

BLOOM FILTERS

Bloom filters address the classical problem of storing keys using hash functions, except that keys are **NOT** stored.

[- Set S of n keys

[- Goal: check for membership query " $x \in S$ " **without storing S**
(just to clarify: we cannot list the keys in S)
with Bloom filters

Clearly we cannot be deterministic

$\Rightarrow$ randomized with BOUNDED error probability

(eg proxy caching, a web page is reloaded if a mistake occurs)

IDEA

- ▶ Set S of n keys, $S \subseteq U$ (universe)
- ▶ Randomly choose $h_1, h_2, \dots, h_k \in \mathcal{H}$ independent $\wedge$ $\mathcal{H}$ universal hash functions
↳ $U \rightarrow [m]$
(recall that given $x \in U, y \in [m] : \Pr[h_i(x) = y] = \frac{1}{m}$)
- ▶ GOAL: check " $x \in S$ "
 - without storing S
 - space efficient
 - false positives ($x \notin S$ but we claim that x belongs to S)

Montecarlo algorithm: $x \in S \Rightarrow$ we surely answer YES

$x \notin S \Rightarrow$ we can answer both YES NO

error: low probability

How to build BFs

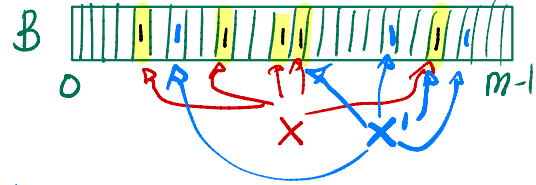
① B = binary vector of m bits of space (initially, they are all 0s)

② for each $x \in S$ do:

for each $i \in [1..k]$: $B[h_i(x)] = 1$

③ "Destroy" S

maybe it's already 1, but we don't care



obs step 2 is incremental, so we can update B if new keys arrives in S

How to query BFs

① $x \in U$ is query key: want to know if $x \in S \subseteq U$

② return $B[h_1(x)] \wedge B[h_2(x)] \wedge \dots \wedge B[h_k(x)]$ (*)



obs $x \in U$ is not randomly chosen, actually a malicious adversary can choose it to put us into difficulty

Why does it work?

→ Error: $x \notin S$ but $(*)$ holds : minimize its probability

We have built $B[S]$

Q1: For any position $q \in [m]$: $\Pr[B[q]=1]$

Q1': Fix h_i : $\Pr = \frac{1}{m}$

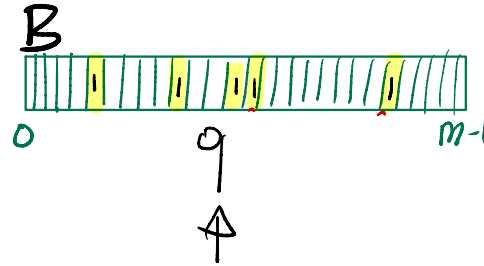
$\overline{Q1}$: $\Pr[B[q]=0 \mid h_i \in H] = 1 - \frac{1}{m}$

$\overline{Q1}$: $\Pr[B[q]=0] = \left(1 - \frac{1}{m}\right)^k$

this is for one key $x \in S$

the correct form is

$\Pr[B[q]=0 \mid x \in S]$



(as we have k independent events,
each with $\Pr = (1 - \frac{1}{m})$)

This is the final probability as we have to go through all $x \in S$

$$\text{Given } q \in [m] : \Pr[B[q]=0] = \left(\underbrace{\left(1 - \frac{1}{m}\right)^k}_{\text{just for one key in } S} \right)^n$$

this for all keys in S

$$p' = \left(1 - \frac{1}{m}\right)^{nk} \leadsto B[q]=0$$
$$(1-p') \leadsto B[q]=1$$

$$\Pr[(*)] = (1-p')^k \quad \text{error probability}$$

minimize

What's the best choice of k ?

It should minimize $(1-p')^k$ when $p' = (1 - \frac{1}{m})^{nk}$

- $(1+y) \sim e^y$ (Taylor's expansion)

- $p' = (1 - \frac{1}{m})^{nk} \sim (e^y)^{nk} = e^{-\frac{nk}{m}} \stackrel{\text{def}}{=} p$
 $y = -\frac{1}{m}$

- we thus minimize $(1-p)^k$ (easier than $(1-p')^k$)

take the log (as it is monotone)

$$\ln(1-p)^k = k \ln(1-p)$$

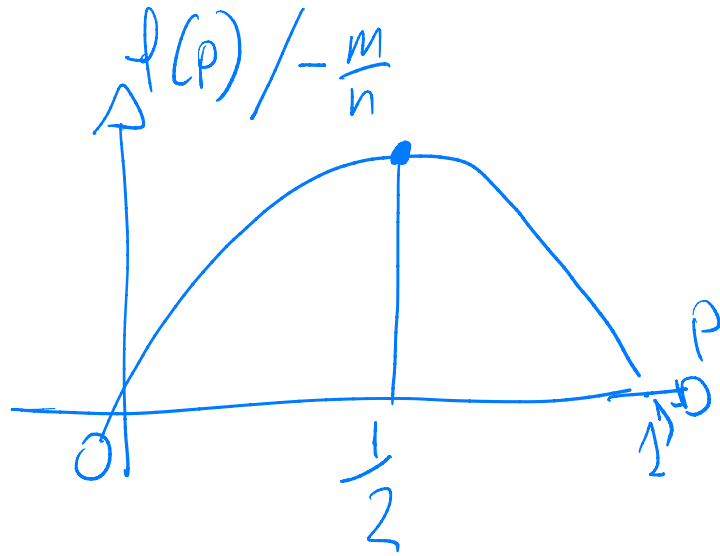
$$\ln p = \ln e^{-\frac{nk}{m}} = -\frac{nk}{m} \Rightarrow$$

$$k = -\frac{m}{n} \ln p$$

$$f(p) = \overbrace{-\frac{m}{n} \ln p}^k \ln(1-p)$$

we minimize $f(p) = -\frac{m}{n} \ln p \ln(1-p)$

$\Rightarrow f(p)$ is minimized
when $p = \frac{1}{2}$



$$\Pr(\star) \sim (1-p)^k = \frac{1}{2^k} = \frac{1}{2^{\frac{m}{n} \ln 2}} = \underbrace{\left(2^{-\ln 2}\right)^{\frac{m}{n}}}_{0.618...} = (0.618...)^{\frac{m}{n}}$$

$$k = -\frac{m}{n} \ln p = -\frac{m}{n} \ln \frac{1}{2} = \frac{m}{n} \ln 2$$

Optimal choice for k is $\frac{m}{n} \ln 2$

and this gives an error probability of $(0.618)^{\frac{m}{n}}$

What about space?

store k hash functions : k pairs of integers
store binary vector B : m bits

} BFs are convenient
When this quantity is
much smaller than the
number of bits needed
to store S

For Bloom filters, the probability of failure is $p = (1-p)^k = \left(\frac{1}{2}\right)^{\frac{m}{n} \ln 2}$

Let's continue the space analysis and see how many bits we need in terms of p

take the logs $\Rightarrow \ln p = \ln \left(\left(\frac{1}{2}\right)^{\frac{m}{n} \ln 2} \right)$

$$\Rightarrow \ln p = \frac{m}{n} \ln 2 \ln \frac{1}{2} \Rightarrow$$

$$\Rightarrow m = \frac{n \ln p}{\ln 2 \ln \frac{1}{2}} = \frac{n \log_2 p}{\ln \frac{1}{2}} = \frac{n \log_2 \frac{1}{p}}{\ln 2} \sim$$

bits per key
 $1.44 \frac{\log_2}{n}$